

Fundación Fulgor
PROCOM

Informe

Laboratorio N°1: Verilog

Autor: Alfici, Facundo Ezequiel

Documento: 44012327

Docentes: Ariel Pola, Santiago Garaventta Pascual y Agustín Pesce

22 de junio de 2026

Índice

1. Introducción	3
2. Desarrollo	4
2.1. Apartado de Código	4
2.2. Apartado de Implementación	5
2.2.1. ¿Cómo se instancian los módulos VIO e ILA y cómo se tra- baja con ellos?	5
2.2.2. Verificación por Hardware	6
3. Conclusión	9

1. Introducción

En este laboratorio se implementó un código de **Verilog** encargado de realizar un desplazamiento por medio de un shift register. Éste desplazamiento se observaba de forma visual sobre la placa, donde la idea fue encender cierto LED cada vez que el desplazamiento se presentaba.

El desplazamiento se realizó de forma circular, utilizando una variable de 4 bits llamada **i_sw**, que representaba los switches de la placa física. Con esta variable, se manejó ciertos parámetros del desplazamiento. Se listan a continuación los parámetros a controlar:

- **Enable:** Mediante el `i_sw[0]`, se habilita o deshabilita el contador. En estado cero '0' se detiene el funcionamiento, mientras que en estado uno '1' se activa el desplazamiento. Es importante destacar que en caso de deshabilitar el contador, éste mantendrá su estado actual, de igual manera que ocurre con el shift register.
- **Límite de conteo:** Utilizando los bits `i_sw[2:1]`, se permite realizar una selección del límite superior de conteo. Contando con 4 valores posibles.
- **Color del led:** El bit `i_sw[3]` se encarga de seleccionar el color del LED a encender durante el desplazamiento.

2. Desarrollo

El código se desarrolló utilizando el software **Vivado 2024.2**, donde el proceso se dividió en 2 partes;

1. **Apartado de Código:** Creación de jerarquía, escritura del código y simulación por medio de un testbench.
2. **Apartado de Implementación:** Adición de constraint, síntesis, implementación, bitstream y conexión por hardware.

Estas partes se detallarán a continuación, comenzando con el primer proyecto realizado.

2.1. Apartado de Código

En este apartado, se realizó la escritura del código, mediante lo visto y propuesto durante las clases. Para ello, se utilizó una estructura jerárquica donde se dividió el código en submódulos. Dentro de estos submódulos, se encuentran el módulo del contador, denominado `count.v`, el módulo del shift register, denominado `shiftreg.v` y el módulo principal que contiene a ambos, llamado `topModule.v`.

La jerarquía se puede ver en la siguiente imagen.

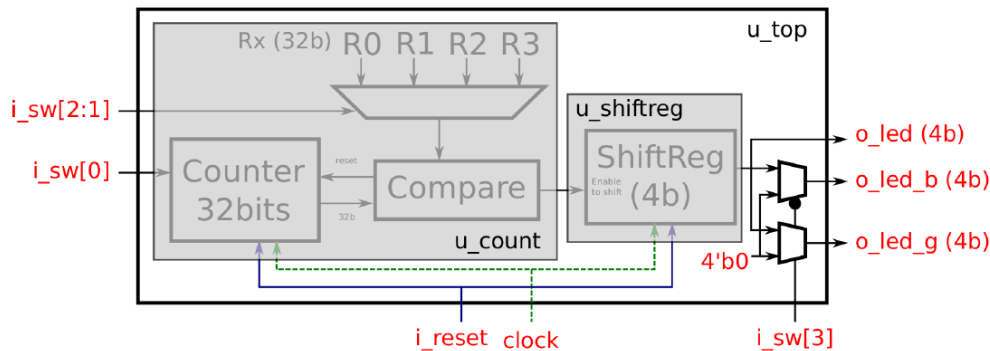


Figura 1: Diagrama jerárquico del proyecto.

Con los códigos terminados, se procedió a realizar la simulación por medio del testbench generado en clase, llamado `tb_topModule.v`. El código generará estímulos para cada uno de las variaciones posibles de los parámetros existentes. Sabiendo esto, en la siguiente imagen se muestra el comportamiento del diseño, ignorando los IP cores VIO e ILA en primera instancia.

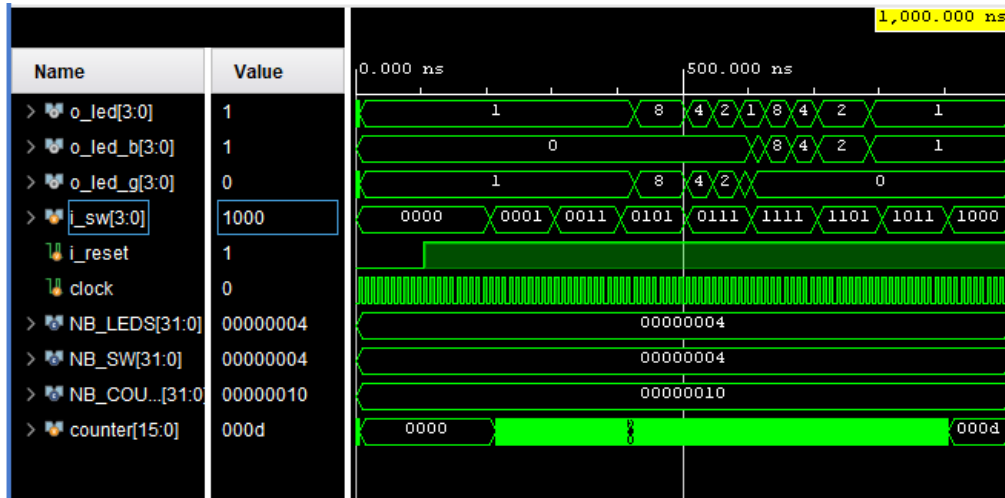


Figura 2: Testbench del topModule.

El testbench tiene una linea de flujo de cambios, donde de manera principal se generan cambios en el parámetro `i_sw` para poder ver el cambio en los valores del contador.

$$i_sw = 0000 \rightarrow i_sw = 0001 \rightarrow i_sw = 0011 \rightarrow i_sw = 0101 \rightarrow i_sw = 0111 \rightarrow i_sw = 1111 \rightarrow i_sw = 1101 \rightarrow i_sw = 1011 \rightarrow i_sw = 1000$$

Este flujo de valores se puede observar claramente en la figura Figura 2, donde, para cada valor de `i_sw`, se puede ver el cambio de encendido de led. También se observa la memoria del contador, donde al detener el contado se mantiene el ultimo valor obtenido.

2.2. Apartado de Implementación

Habiendo visto el apartado de código, se procedió a utilizar los códigos propuestos por el trabajo para realizar las pruebas en el hardware de la fundación. La idea principal es utilizar el archivo `shiftleds_muxViolla.v`. Este archivo se diferencia únicamente de `topModule.v` por la implementación de las instancias de los IP cores VIO e ILA, con la idea de que se pueda comprobar el funcionamiento del sistema mediante la lectura de valores virtuales a distancia.

2.2.1. ¿Cómo se instancian los módulos VIO e ILA y cómo se trabaja con ellos?

Lo que respecta a la instancia de los módulos, como ya se vió previamente, la forma de instanciarlos es utilizando las puntas de prueba definidas utilizando el

IP Integrator de Vivado, el cual nos permite generar un bloque de código con la instancia deseada. Para crear estos bloques se siguió el paso a paso propuesto en la consigna.

El **VIO (Virtual Input/Output)** es un módulo que permite enviar señales virtuales a la FPGA, lo que es útil para probar el funcionamiento del diseño sin necesidad de utilizar hardware físico. Para éste se crearon 3 puntas de prueba de entrada y 3 de salida, las cuales representan los leds como entradas y el selector, reset y switch como salidas.

Por otra parte, el **ILA (Integrated Logic Analyzer)** es un módulo que permite analizar señales internas de la FPGA en tiempo real, lo que es útil para depurar el diseño y verificar su funcionamiento. Para este otro caso, se creó 1 punta de prueba de entrada que analizará el comportamiento de los leds.

La forma en la que interactuarán estos módulos con el resto de parámetros del código será mediante variables wire, de la misma manera que se utilizaría con cualquier otro módulo. Sabiendo esto, se creó un wire que representará un reset externo, los switches y el selector sel multiplexor de los switches. Estas conexiones se pueden ver en la siguiente figura.

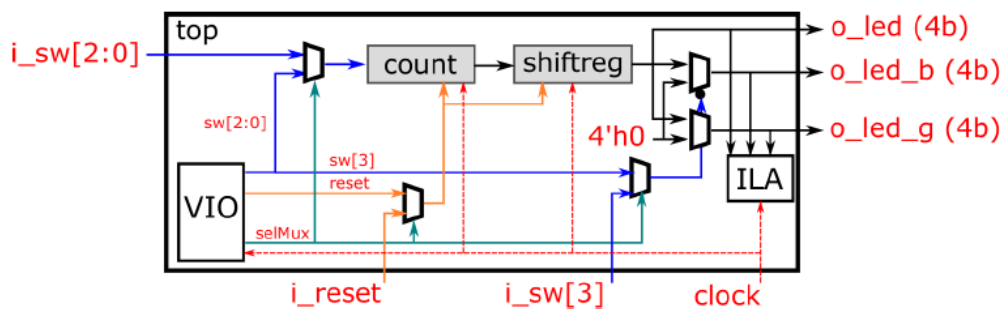


Figura 3: Diagrama de implementación del VIO e ILA.

2.2.2. Verificación por Hardware

Mediante el protocolo SSH (Secure Shell) se realizó la conexión a la FPGA de forma remota. Esto utilizando la siguiente línea de código desde una terminal.

```
Terminal
1  ssh -L 3121:172.16.0.231:3121 user@181.231.212.3 -p 2222
```

Con este comando la PC se conecta directamente al servidor de la fundación y activa el puerto 3121, siendo este el puerto que utiliza Vivado por defecto en el Hardware Manager. Una vez dentro, se simuló el comportamiento y se visualizó la salida mediante el Logic Analyzer.

Teniendo en cuenta que el cambio de valor de la salida **o_led**, se muestran a continuación cada uno de los valores posibles en el analizador.

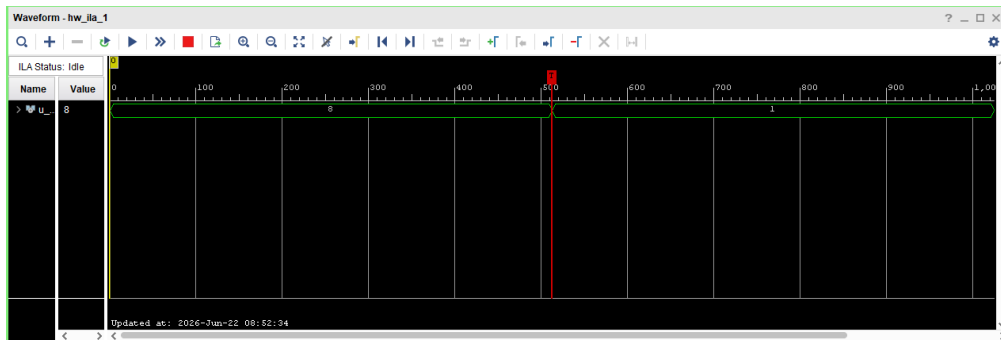


Figura 4: Waveform de R3 en ILA.

Como para el límite superior R3 del contador visto en la figura Figura 4 la cantidad de cambios visibles se reduce, lo que se plantea es cambiar el límite a un valor menor para verificar el funcionamiento. Según esto, se tienen las siguientes capturas.

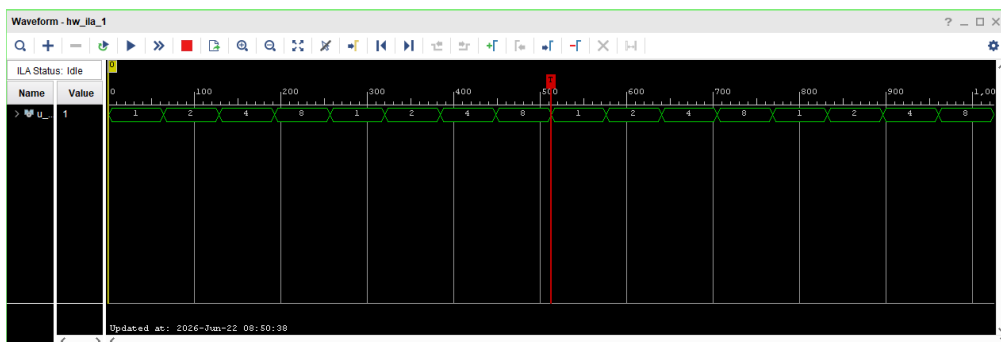


Figura 5: Waveform de R0 en ILA.

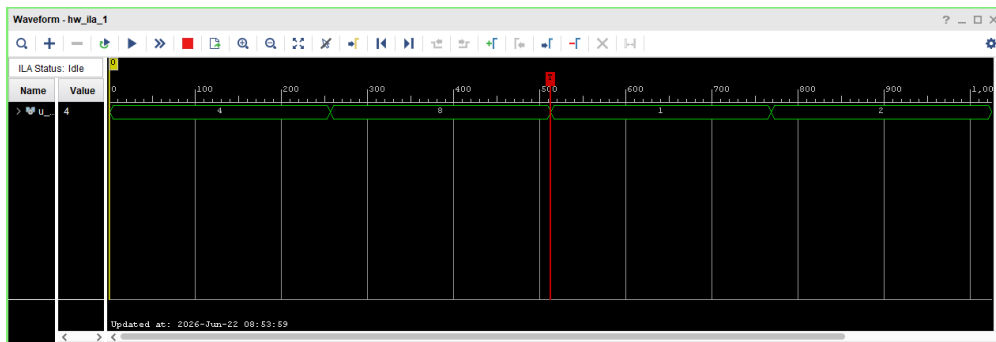


Figura 6: Waveform de R2 en ILA.

Utilizando el límite superior **R2** para una cantidad de bits de contador de 16, se puede ver perfectamente el funcionamiento del sistema.

3. Conclusión

Dadas las capturas, se concluye que la verificación en FPGA remota fue exitoso, debido que el comportamiento de la salida es comparable con el funcionamiento del Testbench realizado.

El principal problema presentado en el proceso de prueba se dió en la **Verificación por Hardware**, donde se tuvo que buscar el valor de bits correcto para el contador, dado que al utilizar un valor muy alto, aumentaba en gran medida la resolución y no permite ver los cambios de estado correctamente. Se optó por utilizar 16b para este valor.